Проектирование микропроцессорной системы на базе МК i8051

Описание программной части

Листов 29

Выполнил: Судьбинин Н.А.
Группа: Аэ-21-23
Вариант: 11

Проверил: Михалин С.Н.

Москва, 2026

Подп. и дата
Инв. № дубл.
Взам. инв. №
Подп. и дата
Инв. № подл.

АННОТАЦИЯ

В данном документе приведено описание программы «Программа управления объектом», предназначенной для управления объектом микропроцессорной системой (МПС) на базе микроконтроллера AT89S53: ввода данных с клавиатуры и от внешнего устройства, формирования управляющих сигналов Y_1 и Y_2 и вывода информации на семисегментный индикатор.

Для функционирования данной программы необходима МПС, собранная по схеме электрической принципиальной варианта 11 (пояснительная записка по аппаратной части).

Исходным языком программы «Программа управления объектом» является ассемблер микроконтроллеров семейства MCS-51.

Основными функциями программы являются: инициализация МПС, сканирование клавиатуры, работа с кольцевым буфером на 1910 байт, выработка строба Y_1 длительностью 313 мс, выработка сигнала Y_2 со стробом 1751 мкс, индикация на одном разряде.

Оформление программного документа «Описание программы» произведено по требованиям ЕСПД (ГОСТ 19.101-77, ГОСТ 19.103-77, ГОСТ 19.104-78, ГОСТ 19.105-78, ГОСТ 19.106-78, ГОСТ 19.402-78, ГОСТ 19.604-78).

СОДЕРЖАНИЕ

1. Общие сведения..... 4
2. Функциональное назначение..... 5
2.1. Классы решаемых задач..... 5
2.2. Назначение программы..... 5
2.3. Сведения о функциональных ограничениях на применение..... 5
3. Описание логической структуры..... 6
3.1. Алгоритм программы..... 6
3.2. Структура программы с описанием функций составных частей и связи между ними..... 19
3.3. Связи программы с другими программами..... 20
4. Входные данные..... 21
4.1. Клавиатура..... 21
4.2. Сигнал X2..... 21
5. Выходные данные..... 22
5.1. Характер и организация выходных данных..... 22
5.2. Формат, описание и способ кодирования выходных данных..... 22
ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ..... 24
ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ..... 25
ПРИЛОЖЕНИЕ А. Листинги программ..... 26

1. ОБЩИЕ СВЕДЕНИЯ

Обозначение и наименование программы — «Программа управления объектом», вариант 11, группа Аэ-21-23.

Программа низкого уровня не требует для своей работы иных программ. Исходным языком программирования является ассемблер MCS-51 (синтаксис Intel ASM51, совместимый с Keil A51 и MCU 8051 IDE).

Среда разработки и инструментальные средства: интегрированная среда Keil µVision V5 (Keil Software, пакет PK51); ассемблер — A51; компоновщик — BL51.

2. ФУНКЦИОНАЛЬНОЕ НАЗНАЧЕНИЕ

2.1. Классы решаемых задач

- инициализация МПС;
- сканирование клавиатуры — ввод сигнала X_1 (число, введённое с клавиатуры);
- формирование сигнала Y_1 (строб заданной длительности);
- ввод сигнала X_2 (сохранение данных в кольцевом буфере);
- формирование сигнала Y_2 и сопровождающего строба;
- вывод символа на индикатор.

2.2. Назначение программы

МПС принимает информацию $\{X\}$ об объекте управления от датчиков, вырабатывает управляющие воздействия $\{Y\}$ в соответствии с заданным алгоритмом управления и подаёт их на исполнительные механизмы. Программа разбита на подзадачи по функциональному признаку:

- инициализация МПС;
- обработка прерываний: опрос клавиатуры, запись в буфер, таймеры генераторов сигналов Y;
- определение кода нажатой клавиши;
- вывод данных на семисегментный индикатор;
- формирование сигнала Y_2;
- обеспечение работы кольцевого буфера (запись и чтение отсчётов X_2).

В соответствии с вариантом (табл. 1 ТЗ) разработан код трёх программ:

- программа 1 — обработчик прерывания клавиатуры (сканирование матрицы), код нажатой клавиши в аккумуляторе или 0FFh;
- программа 2 — запись в кольцевой буфер (данные через аккумулятор) с управлением флагами пустоты и переполнения;
- программа 3 — расчёт и выдача сигнала Y_2 со стробом длительностью T_2 (прерывание таймера).

2.3. Сведения о функциональных ограничениях на применение

В качестве основы для построения МПС выбран микроконтроллер семейства MCS-51 — AT89S53, тактовая частота 12 МГц (машинный цикл 1 мкс). Таймер 2 и сторожевой таймер не используются (совместимость с i8051).

Интерфейсы:

- входные данные X_2 — 8-разрядное целое без знака по независимой параллельной шине со стробом записи низкого уровня в двухпортовое ОЗУ, нерегулярно, до 4 отсчётов в секунду;
- входные данные X_1 — десятичная цифра 0–9 с матричной клавиатуры 3×4 (сканирование по столбцам);
- выходной сигнал Y_2 — 8-разрядный параллельный код в выходном регистре со стробом низкого уровня длительностью 1751 мкс на выводе P1.5;
- выходной сигнал Y_1 — строб низкого уровня длительностью 313 мс на выводе P1.4;
- индикация — один семисегментный индикатор (общий катод), время индикации символа не более T_3 = 2,745 с.

3. ОПИСАНИЕ ЛОГИЧЕСКОЙ СТРУКТУРЫ

3.1. Алгоритм программы

После сброса программа один раз выполняет инициализацию (рисунок 1): ставит стек на 07h, включает дешифратор адреса (CS_EN = 1), опускает все столбцы клавиатуры, гасит индикатор, переводит стробы Y1 и Y2 в пассивную единицу, делает буфер пустым (голова = хвост = E000h, F_EMPTY = 1, F_OVF = 0), настраивает оба таймера в режим 1 и разрешает прерывания INT0, INT1, Timer0 и Timer1. Дальше работает основной цикл (рисунок 2): если введена ненулевая цифра X1, в буфере есть отсчёт и прошлый строб Y2 уже закончился, он берёт X2 из буфера и выдаёт новое Y2; если X1 = 0, один раз выдаётся Y2 = 0.

Всё, что зависит от внешних событий, сделано в прерываниях. INT0 (по спаду) — нажатие клавиши: опрос матрицы KbScan (рисунок 3) и реакция на код (рисунок 4). INT1 (тоже по спаду) — строб внешнего устройства, которое уже записало X2 правым портом в ячейку обмена 0FFFFh: обработчик дожидается конца строба, читает эту ячейку левым портом и кладёт отсчёт в кольцо по адресу «головы» (рисунки 5, 6). Timer0 тикает каждую 1 мс и отсчитывает длительность строба Y1 (313 тиков) и время индикации (2745 тиков), Timer1 отмеряет строб Y2 одним отсчётом на 1751 мкс и в обработчике снимает его (рисунок 13). Благодаря этому ни одна процедура не ждёт в цикле, и основной цикл всегда свободен.



Рис. 1. Схема алгоритма: Инициализация МПС и основная программа

Схема «Инициализация МПС и основная программа» выполняется так: вход — «Сброс МК»; SP := 07h; CS_EN := 1 (разрешить дешифратор адреса); начальные значения переменных и выводов (стробы в 1, буфер пуст, X1 := 0); обмен с устройством: все столбцы клавиатуры := 0; индикатор погасить; режимы таймеров (TMOD) и прерываний; разрешить прерывания; ожидание прерываний (jmp $).

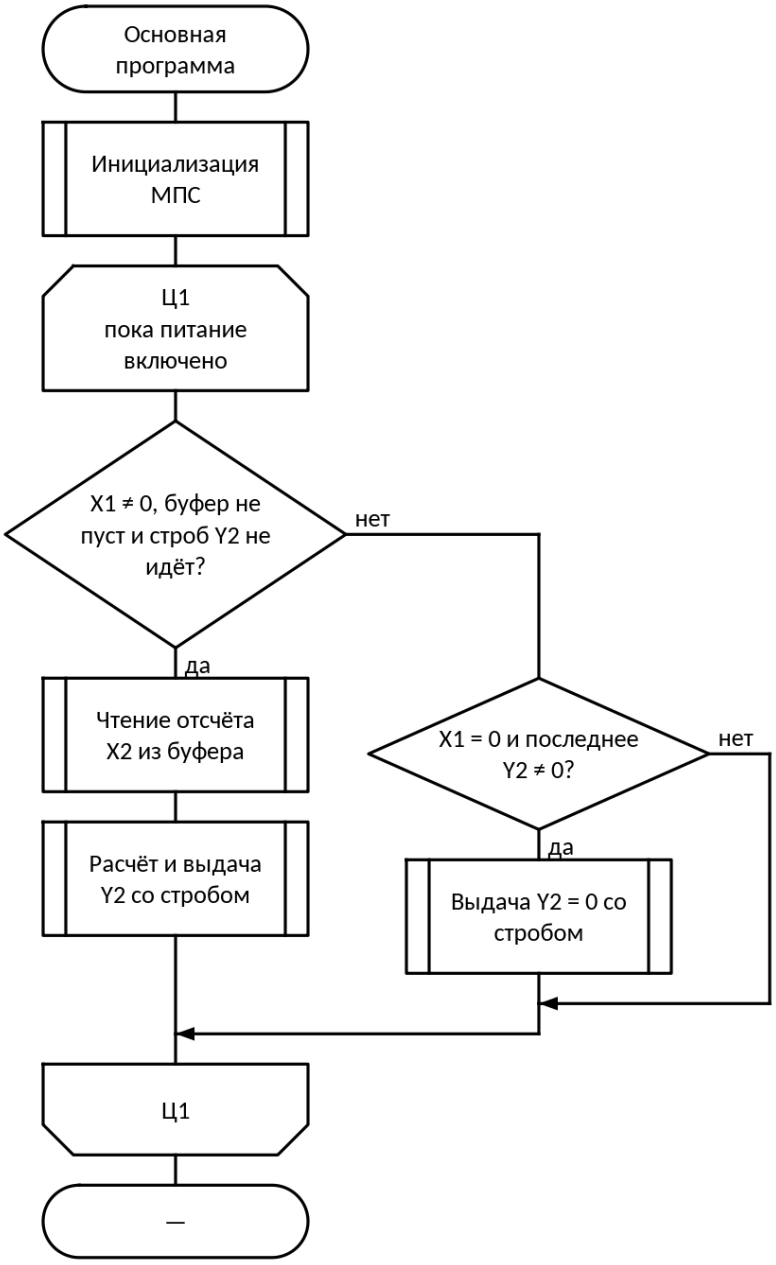


Рис. 2. Схема алгоритма: Основной цикл программы

Схема «Основной цикл программы» выполняется так: вход — «Основная программа»; вызывается подпрограмма: инициализация МПС; в цикле Ц1 (пока питание включено) повторяется: проверяется условие «X1 ≠ 0, буфер не пуст и строб Y2 не идёт»: если да — вызывается подпрограмма: чтение отсчёта X2 из буфера, вызывается подпрограмма: расчёт и выдача Y2 со стробом, иначе — проверяется условие «X1 = 0 и последнее Y2 ≠ 0»: если да — вызывается подпрограмма: выдача Y2 = 0 со стробом.

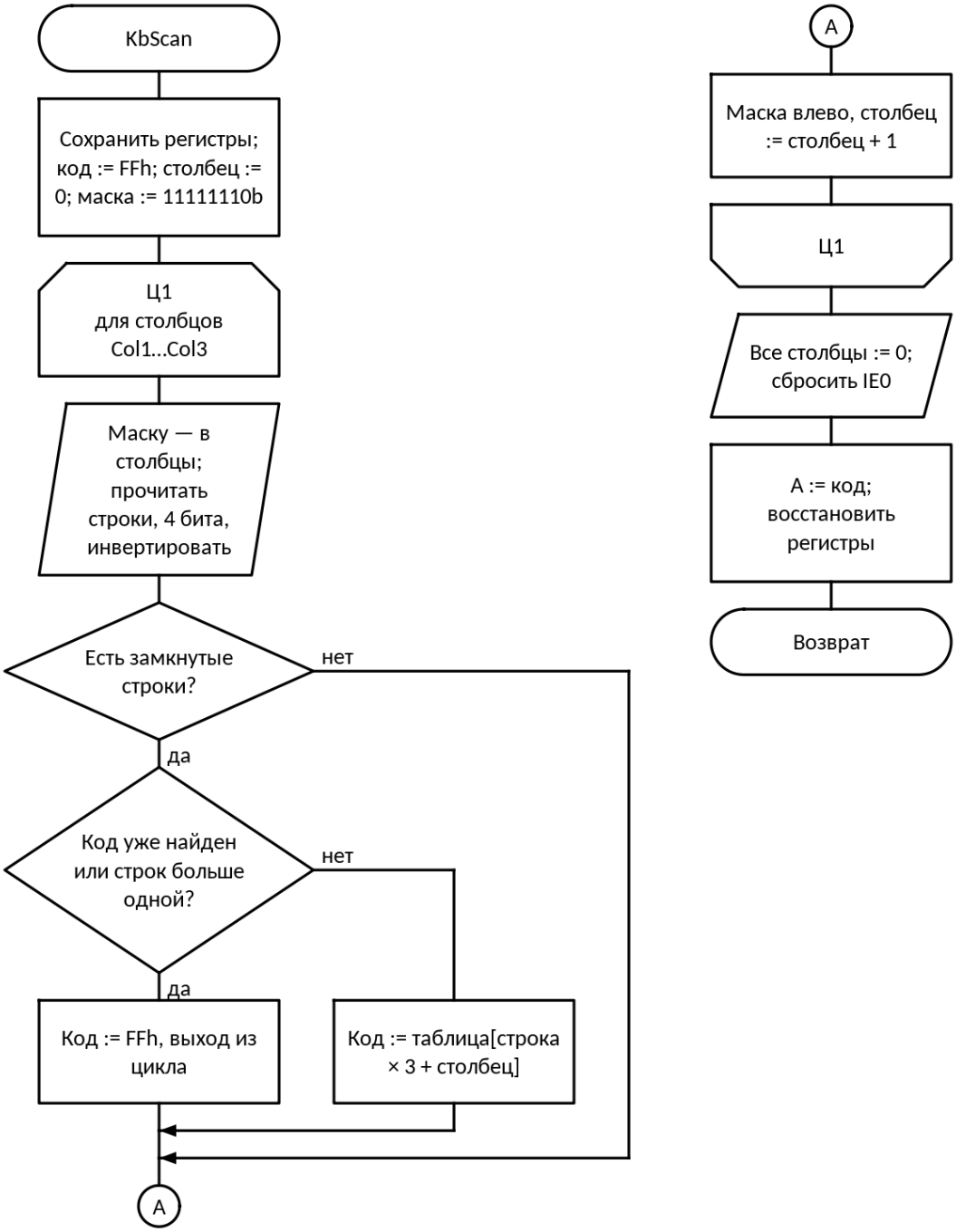


Рис. 3. Схема алгоритма: Опрос клавиатуры KbScan (обработчик INT0)

Опрос идёт по столбцам: в регистр столбцов пишется маска с единственным нулём, из буфера строк читаются четыре младших бита (старшие разряды шины не подключены, поэтому маскируются), и после инверсии единица означает замкнутую строку. Если в столбце замкнута одна строка и раньше ничего не находили, код берётся из таблицы по индексу строка × 3 + столбец; при второй найденной клавише опрос прекращается с результатом FFh. В конце все столбцы снова опускаются в ноль, иначе следующее нажатие не вызовет INT0, и сбрасывается IE0, который взводился, пока линия дёргалась во время перебора столбцов.

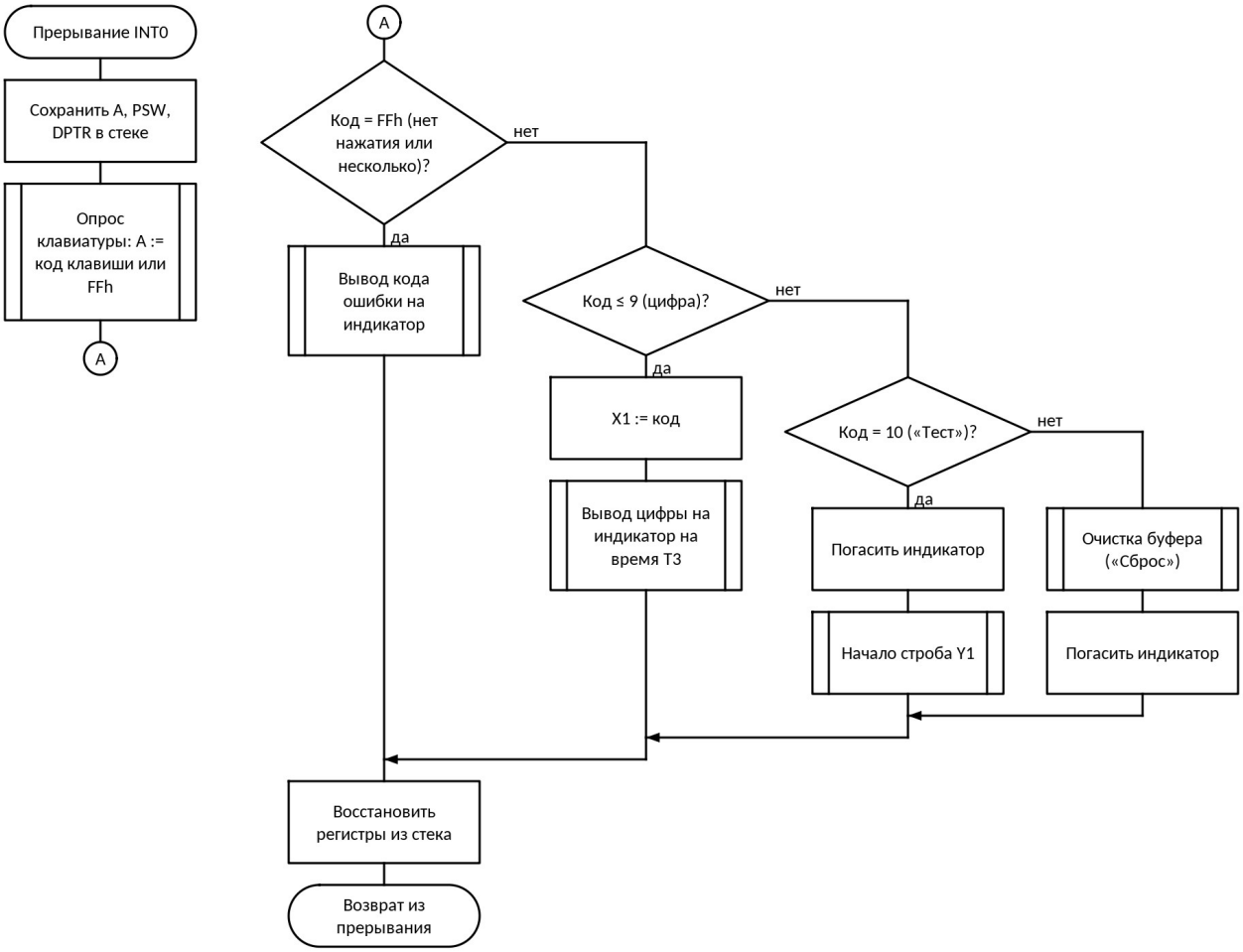


Рис. 4. Схема алгоритма: Обработчик прерывания INT0: реакция на клавишу

Схема «Обработчик прерывания INT0: реакция на клавишу» выполняется так: вход — «Прерывание INT0»; сохранить A, PSW, DPTR в стеке; вызывается подпрограмма: опрос клавиатуры: A := код клавиши или FFh; проверяется условие «Код = FFh (нет нажатия или несколько)»: если да — вызывается подпрограмма: вывод кода ошибки на индикатор, иначе — проверяется условие «Код ≤ 9 (цифра)»: если да — X1 := код, вызывается подпрограмма: вывод цифры на индикатор на время T3, иначе — проверяется условие «Код = 10 («Тест»)»: если да — погасить индикатор, вызывается подпрограмма: начало строба Y1, иначе — вызывается подпрограмма: очистка буфера («Сброс»), погасить индикатор; восстановить регистры из стека; в конце — возврат из прерывания.

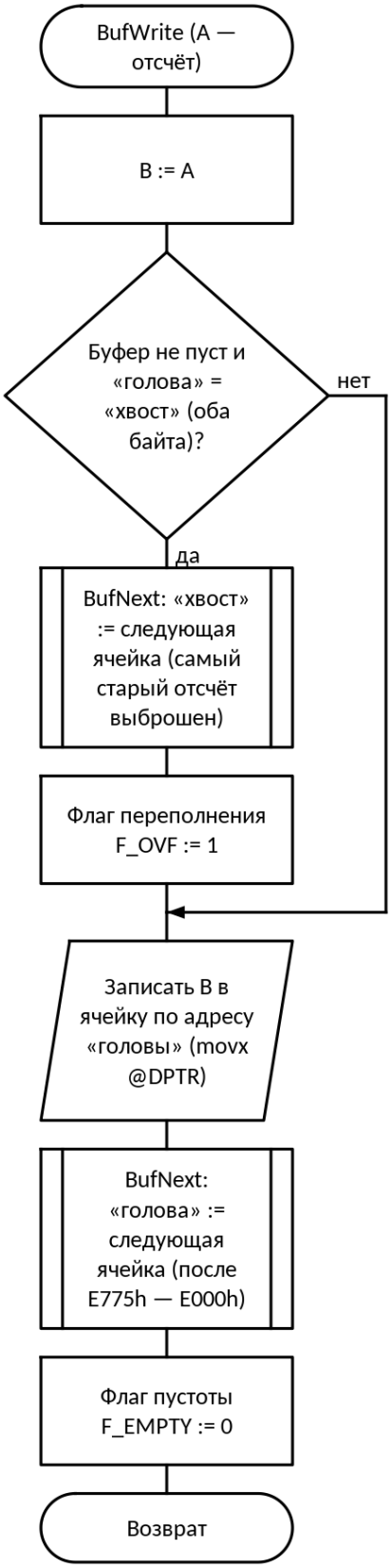


Рис. 5. Схема алгоритма: Запись отсчёта в кольцевой буфер BufWrite

Отсчёт сначала откладывается в B, потому что аккумулятор нужен для сравнения указателей. Буфер считается полным, когда он не пуст и «голова» совпала с «хвостом» по обоим байтам адреса, то есть заняты все 1910 ячеек; тогда «хвост» сдвигается на одну ячейку (самый старый отсчёт теряется) и ставится флаг переполнения. Затем отсчёт пишется в ячейку

«головы», «голова» сдвигается подпрограммой BufNext, которая после адреса E775h возвращает указатель на E000h, и сбрасывается флаг пустоты.

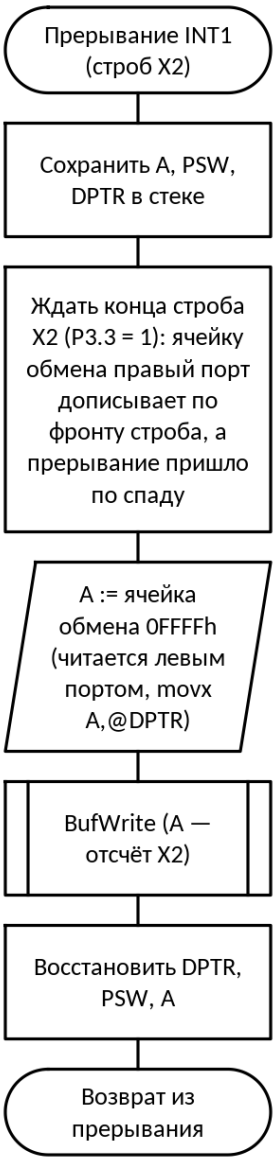


Рис. 6. Схема алгоритма: Обработчик прерывания INT1: приём отсчёта X2 от внешнего устройства

Сам отсчёт в память кладёт внешнее устройство: по своему стробу оно пишет X2 правым портом IDT7005 в ячейку обмена 0FFFFh, а МК в этот момент только получает прерывание. Обработчик сохраняет A, PSW и DPTR, потому что прерывание может прийти посреди расчёта Y2 или чтения буфера. Дальше он крутится на проверке P3.3, пока строб не вернётся в единицу: INT1 срабатывает по спаду, а байт в ячейке окончательно фиксируется только по фронту строба, и прочитать её раньше — значит взять старое значение. После этого ячейка обмена читается левым портом в аккумулятор и передаётся процедуре записи (рисунок 5), которая уже двигает «голову» и при необходимости «хвост». В конце регистры восстанавливаются в обратном порядке.

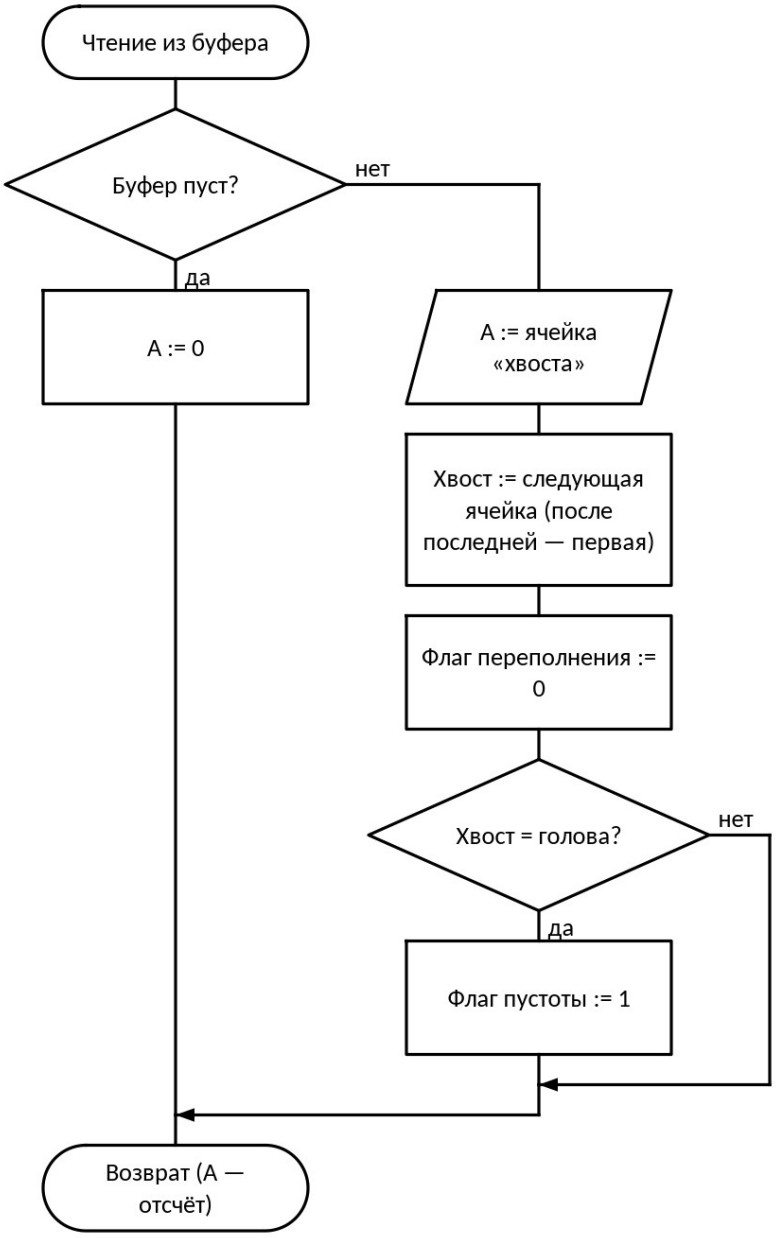


Рис. 7. Схема алгоритма: Чтение отсчёта из кольцевого буфера

Схема «Чтение отсчёта из кольцевого буфера» выполняется так: вход — «Чтение из буфера»; проверяется условие «Буфер пуст»: если да — A := 0, иначе — обмен с устройством: A := ячейка «хвоста», хвост := следующая ячейка (после последней — первая), флаг переполнения := 0, проверяется условие «Хвост = голова»: если да — флаг пустоты := 1; в конце — возврат (A — отсчёт).



Рис. 8. Схема алгоритма: Очистка кольцевого буфера (клавиша «Сброс»)

Схема «Очистка кольцевого буфера (клавиша «Сброс»)» выполняется так: вход — «Очистка буфера»; запретить прерывание INT1; хвост := голова; флаг пустоты := 1; флаг переполнения := 0; разрешить прерывание INT1; в конце — возврат.

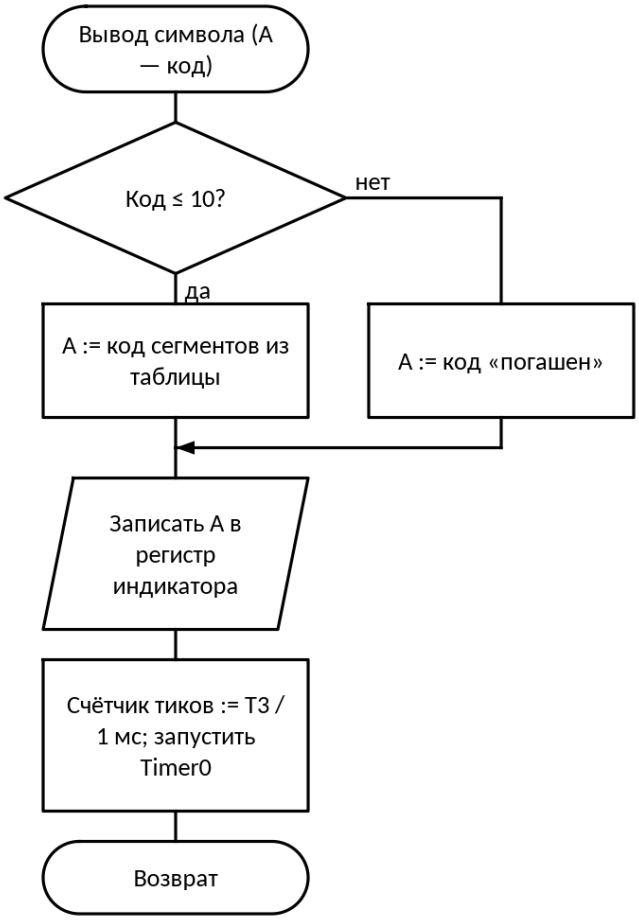


Рис. 9. Схема алгоритма: Вывод символа на индикатор

Схема «Вывод символа на индикатор» выполняется так: вход — «Вывод символа (A — код)»; проверяется условие «Код ≤ 10»: если да — A := код сегментов из таблицы, иначе — A := код «погашен»; обмен с устройством: записать A в регистр индикатора; счётчик тиков := ТЗ / 1 мс; запустить Timer0; в конце — возврат.

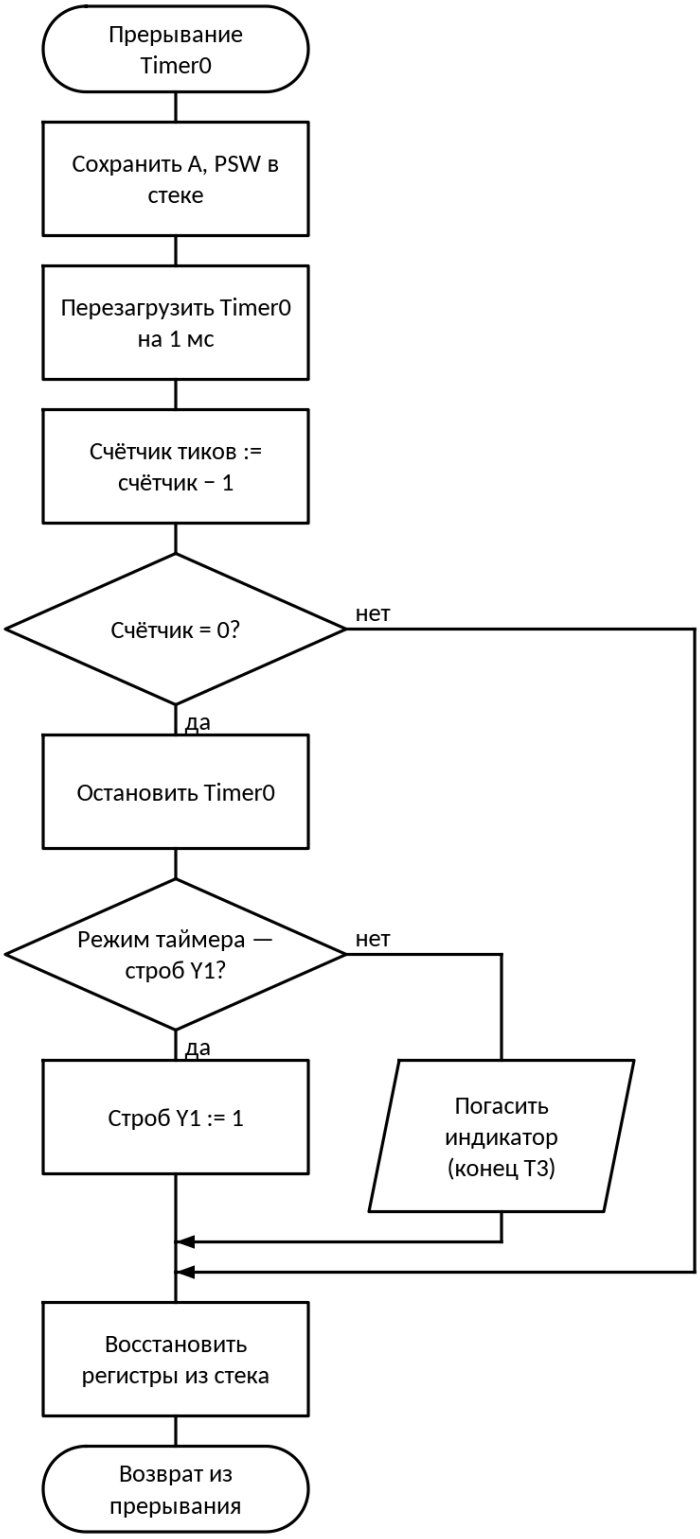


Рис. 10. Схема алгоритма: Обработчик прерывания Timer0: отсчёт T1 и T3

Схема «Обработчик прерывания Timer0: отсчёт T1 и T3» выполняется так: вход — «Прерывание Timer0»; сохранить A, PSW в стеке; перезагрузить Timer0 на 1 мс; счётчик тиков := счётчик − 1; проверяется условие «Счётчик = 0»: если да — остановить Timer0, проверяется условие «Режим таймера — строб Y1»: если да — строб Y1 := 1, иначе — обмен с устройством: погасить индикатор (конец T3); восстановить регистры из стека; в конце — возврат из прерывания.



Рис. 11. Схема алгоритма: Начало строба Y1

Схема «Начало строба Y1» выполняется так: вход — «Строб Y1»; счётчик тиков := T1 / 1 мс; Timer0 := тик 1 мс; строб Y1 := 0; запустить Timer0; в конце — возврат.

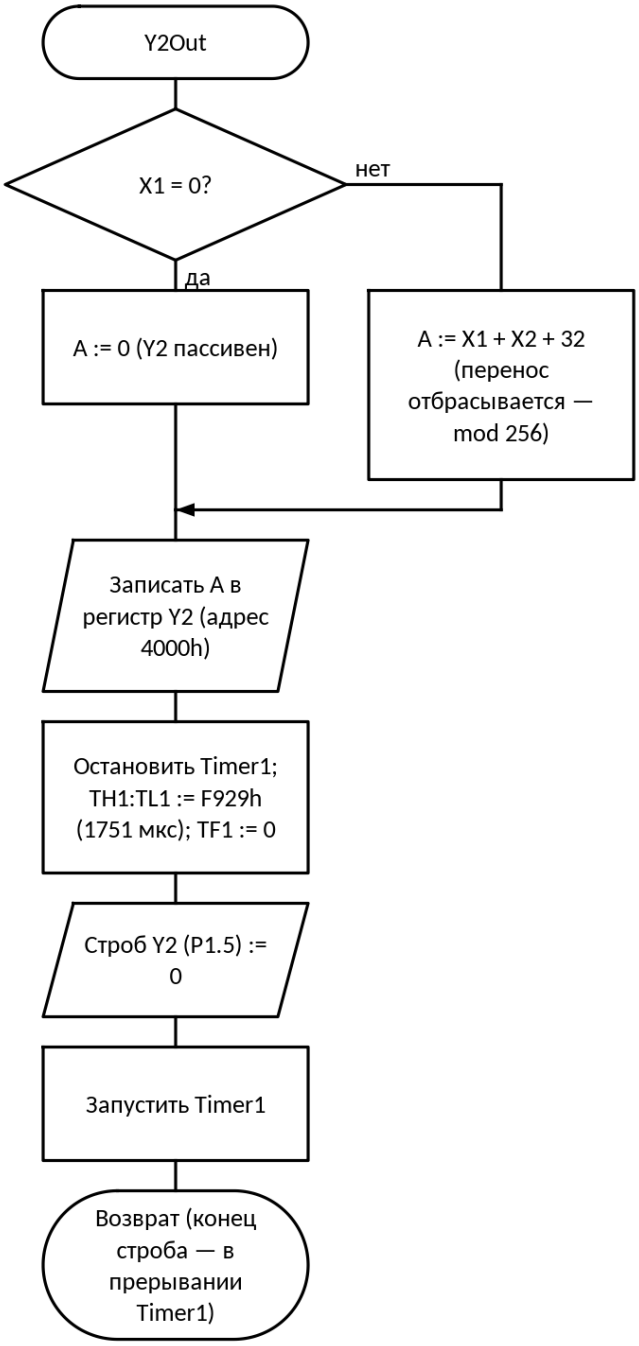


Рис. 12. Схема алгоритма: Расчёт и выдача сигнала Y2 (Y2Out)

Если X1 = 0, на выход идёт ноль, иначе в аккумуляторе складываются X1, X2 и константа G + M = 32; перенос из старшего разряда просто теряется, это и даёт остаток по модулю 256. Результат сначала записывается в регистр Y2, и только потом строб на P1.5 опускается в ноль — так к началу строба на выходе регистра уже правильные данные. Timer1 перед этим останавливается, загружается значением F929h (до переполнения ровно 1751 такт по 1 мкс) и с очищенным TF1 запускается сразу после спада строба; процедура на этом возвращается, не дожидаясь конца строба.



Рис. 13. Схема алгоритма: Обработчик прерывания Timer1: конец строба Y2

Обработчик сначала поднимает строб Y2 в единицу, чтобы задержка входа в прерывание как можно меньше удлиняла импульс, и затем останавливает Timer1 до следующей выдачи Y2. Аккумулятор и PSW здесь не меняются, поэтому сохранять их не нужно.

3.2. Структура программы с описанием функций составных частей и связи между ними

Программа состоит из инициализации, основной программы (ожидание прерываний) и подпрограмм, описанных ниже. Описание каждой подпрограммы взято из заголовочного комментария в исходном тексте (приложение А).

3.2.1 Подпрограмма KbScan (программа 1)

Назначение: опрос матрицы клавиатуры 3x4 (вызывается из обработчика INT0). Столбцы опускаются в 0 по одному, по прочитанным строкам ищется единственная нажатая клавиша.

Входные данные: нет.

Выходные данные: A — код клавиши по табл. 3 T3 (0…9, «T» = 10, «C» = 11) или 0FFh, если не нажато ничего либо нажато две клавиши и больше. После опроса все столбцы снова в 0. Портит только A: PSW, DPTR, B, R2–R5 сохраняются в стеке.

3.2.2 Подпрограмма BufWrite (программа 2)

Назначение: запись отсчёта в кольцевой буфер IDT7005. Буфер полон, когда «голова» догнала «хвост» и он не пуст (политика V: занято все BUF_SIZE ячеек); тогда самый старый отсчёт выбрасывается — «хвост» сдвигается вперёд, ставится F_OVF.

Входные данные: A — отсчёт.

Выходные данные: отсчёт в ячейке бывшей «головы», «голова» на следующей ячейке, F_EMPTY = 0; F_OVF = 1, если затёрт самый старый отсчёт (иначе F_OVF не меняется). Портит A, B, DPTR, PSW.

3.2.3 Подпрограмма BufNext (программа 2)

Назначение: следующая ячейка кольцевого буфера.

Входные данные: DPTR — адрес ячейки буфера.

Выходные данные: DPTR — адрес следующей; за последней ячейкой (BUF_END − 1) идёт первая (BUF_START). Портит A, PSW.

3.2.4 Подпрограмма Y2Out (программа 3)

Назначение: расчёт Y2 = (G + M + X1 + X2) mod 256, при X1 = 0 выдаётся Y2 = 0; значение пишется в регистр Y2, затем запускается строб PIN_Y2 = 0 длительностью T2. Процедура не ждёт конца строба: его снимает обработчик Timer1.

Входные данные: X1, X2 (внутреннее ОЗУ).

Выходные данные: регистр Y2, PIN_Y2 = 0, Timer1 запущен на T2. Портит A, DPTR, PSW.

3.2.5 Обработчик прерывания Timer1 (программа 3)

Назначение: Timer1 досчитал T2 — строб Y2 закончен.

Распределение ресурсов микроконтроллера — прерывания, адреса внешних устройств и переменные — приведено в таблицах 1–3.

Таблица 1 — Прерывания

Источник	**Вектор**	**Назначение**	**Запуск**
INT0 (P3.2)	0003h	нажатие клавиши	по спаду
Timer0	000Bh	отсчёт T_1 (313 мс) и T_3 (2745 мс), тик 1 мс	переполнение
INT1 (P3.3)	0013h	строб записи X_2	по спаду
Timer1	001Bh	отсчёт T_2 (1751 мкс)	переполнение
UART	0023h	не используется (заглушка)	—

Таблица 2 — Адреса внешних устройств (обращение командами movx при P3.4 = 1)

Устройство	**Выход дешифратора**	**Адреса**
Индикатор	Y1	2000h–3FFFh
Клавиатура	Y3	6000h–7FFFh
Буфер (IDT7005)	Y7	0E000h–0FFFFh
Регистр Y2	Y2	4000h–5FFFh

Таблица 3 — Переменные во внутреннем ОЗУ

Имя	**Адрес**	**Назначение**
HEAD_L, HEAD_H	51h, 52h	«голова» — адрес ячейки буфера для следующей записи
TAIL_L, TAIL_H	53h, 54h	«хвост» — адрес ячейки буфера для следующего чтения
F_EMPTY	21h.3 (бит 0Bh)	флаг пустоты буфера
F_OVF	24h.0 (бит 20h)	флаг переполнения буфера
X1	30h	сигнал X_1 — цифра с клавиатуры
X2	31h	сигнал X_2 — отсчёт из буфера

3.3. Связи программы с другими программами

Программа «Программа управления объектом» в ходе своей работы не использует других программ.

4. ВХОДНЫЕ ДАННЫЕ

4.1. Клавиатура

Клавиатура — матрица 3×4 (3 столбцов × 4 строк), сканирование по столбцам: в регистр столбцов (адрес 6000h, разряды D0…D2) выдаётся 0 в опрашиваемый столбец, строки читаются через повторитель по тому же адресу (разряды D0…D3, 0 — контакт замкнут). Нажатие любой клавиши при всех столбцах в 0 вызывает прерывание INT0. Коды клавиш (табл. 3 ТЗ) и их расположение — в таблице 4.

Таблица 4 — Расположение клавиш и их коды («клавиша» — код)

	Col1	**Col2**	**Col3**
Row1	«1» — 1	«2» — 2	«3» — 3
Row2	«4» — 4	«5» — 5	«6» — 6
Row3	«7» — 7	«8» — 8	«9» — 9
Row4	«0» — 0	«Т» — 10	«С» — 11

Результат опроса — код единственной нажатой клавиши (0–9 — цифры, 10 — «Тест», 11 — «Сброс») или 0FFh, если не нажато ничего или нажато больше одной клавиши. Цифра становится значением X_1 (начальное значение $X_1 = 0$).

4.2. Сигнал X2

X_2 — 8-разрядное целое без знака. Внешнее устройство выставляет отсчёт на разъёме доступа к буферу и сопровождает его стробом, который вызывает прерывание INT1. Буфером управляет МК: обработчик делает movx-запись по адресу «головы», и правый порт двухпортового ОЗУ IDT7005 заносит в ячейку байт X_2 с разъёма. Отсчёты хранятся в кольцевом буфере на 1910 байт (адреса 0E000h–0E775h).

5. ВЫХОДНЫЕ ДАННЫЕ

5.1. Характер и организация выходных данных

Индикация. На индикаторе отображается последняя введённая цифра — до ввода следующей, но не дольше T_3 = 2,745 с; при ошибке — код ошибки с горящей точкой dp. Начальное состояние — индикатор погашен.

Кольцевой буфер во внешнем ОЗУ. Ячейки 0E000h–0E775h. «Голова» — адрес ячейки для следующей записи, «хвост» — для следующего чтения; после последней ячейки следует первая. Буфер пуст — флаг пустоты = 1 (голова = хвост). Если буфер заполнен, новый отсчёт затирает самый старый (хвост сдвигается) и устанавливается флаг переполнения; после чтения флаг переполнения сбрасывается.

Управляющие сигналы. Y_1 — строб низкого уровня длительностью 313 мс по клавише «Тест». $Y_2 = (21 + 11 + X_1 + X_2) \bmod 256$ при $X_1 \neq 0$, иначе $Y_2 = 0$ (пассивное состояние); вырабатывается, пока в буфере есть данные и $X_1 \neq 0$. Каждая запись Y_2 в выходной регистр сопровождается стробом длительностью 1751 мкс; до записи регистр выдаёт предыдущее значение.

Расчётные параметры работы программы. Константы таймеров (кварц 12 МГц, машинный цикл $T_{мц} = 12 / f = 1$ мкс, режим 1 — 16 бит):

TH1:TL1 = 65536 − T2 / T_мц = 65536 − 1751 = 63785 = 0F929h (строб Y2, таймер 1)

TH0:TL0 = 65536 − 1 мс / T_мц = 65536 − 1000 = 64536 = 0FC18h (тик 1 мс, таймер 0)

N_T1 = T1 / 1 мс = 313; N_T3 = T3 / 1 мс = 2745 (число тиков)

Минимальное время заполнения буфера (при $X_1 = 0$ сигнал Y_2 пассивен и буфер не читается, отсчёты поступают с максимальной интенсивностью):

t_min = V / f_X2 = 1910 / 4 = 477,5 с (8,0 мин).

Максимальная глубина стека для программы 1 — 12 байт (SP от 07h до 13h). Значение получено прогоном программы в симуляторе на всех сценариях проверки (каждая клавиша, одновременные нажатия, помеха на INT0) и складывается из адреса возврата при входе в обработчик INT0 (2 байта), адреса возврата при вызове процедуры опроса (2 байта) и сохраняемых в стеке регистров.

5.2. Формат, описание и способ кодирования выходных данных

Код символа (табл. 2 ТЗ) преобразуется в код сегментов по таблице 5: разряды D0…D7 регистра индикатора соответствуют сегментам a, b, c, d, e, f, g, dp; индикатор с общим катодом — сегмент горит при 1.

Таблица 5 — Коды сегментов индикатора (dp g f e d c b a)

Код	**Символ**	**Двоичный**	**Шестн.**
0	«0»	00111111	3Fh
1	«1»	00000110	06h
2	«2»	01011011	5Bh
3	«3»	01001111	4Fh
4	«4»	01100110	66h
5	«5»	01101101	6Dh
6	«6»	01111101	7Dh
7	«7»	00000111	07h
8	«8»	01111111	7Fh
9	«9»	01101111	6Fh
10	«E»	01111001	79h
11 и больше	погашен	00000000	00h

Таблица 6 — Коды ошибок (символ с горящей точкой dp, индикация до нажатия любой клавиши, но не дольше ТЗ)

Индикация	**Смысл**	**Когда выставляется**
«1.»	нажато несколько клавиш	опрос клавиатуры нашёл больше одного замкнутого контакта
«2.»	нажатие не распознано	прерывание пришло, а замкнутых контактов нет
«3.»	переполнение буфера	запись X_2 затёрла непрочитанный отсчёт
«4.»	повторный «Тест»	«Тест» нажат, пока идёт строб Y_1

В программах КМ-3 ошибки обрабатываются так: KbScan возвращает FFh и при нескольких нажатых клавишах, и при прерывании без нажатия, а обработчик INT0 по этому коду выводит ошибку «1.» (ситуацию «2.» по одному коду A от «1.» не отличить, поэтому в моей программе они совпадают); переполнение буфера процедура BufWrite отмечает флагом F_OVF, по которому основной цикл показывает «3.». Код «4.» относится к стробу Y1, который в мои три программы не входит.

Сигнал Y_2 выдаётся 8-разрядным кодом в выходной регистр (адрес 4000h), строб — на выводе P1.5; сигнал Y_1 — на выводе P1.4.

В 16-разрядном режиме (режим 1) таймер 0 перезагружается в обработчике прерывания; задержка входа в прерывание (несколько машинных циклов на тик) компенсируется поправкой к загружаемому значению или учитывается как погрешность длительности. В режиме 2 (8 разрядов с автоперезагрузкой) перезагрузку выполняет сам таймер, а тик набирается программным счётчиком переполнений.

ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ

Таблица 7 — Сокращения

Сокращение	**Расшифровка**
МК	микроконтроллер
МПС	микропроцессорная система
ОЗУ	оперативное запоминающее устройство
ПЗУ	постоянное запоминающее устройство
ЕСПД	единая система программной документации
ТЗ	техническое задание
CS	Chip Select, выбор микросхемы
IRQ	запрос прерывания
SP	указатель стека
DPTR	16-разрядный регистр-указатель данных
dp	десятичная точка индикатора

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Изм.	Номера листов (страниц)				Всего листов (страниц) в докум.	№ докум.	Входящий № сопроводит. докум. и дата	Подп.	Дата
	изменённых	заменённых	новых	аннулиро-ванных					

ПРИЛОЖЕНИЕ А

Листинги программ

Программа 1 — файл «Судьбинин НА-код-1»

```
; Судьбинин Н.А., Аэ-21-23, 11, v1
$INCLUDE (vars.inc)             ; адреса, биты, константы варианта; в файл для робота mpscode вклеит его текст
DSEG AT 30h
; переменных в ОЗУ нет: всё держится в регистрах банка 0 и в стеке
CSEG AT 0h
        sjmp START
org 03h                         ; INT0 — нажатие клавиши: процедура вызывается отсюда
        call KbScan ; %proc%
        reti
org 0bh ; "заглушка" для Tmr0_ovf
        nop
        reti
org 13h ; "заглушка" для INT1
        nop
        reti
org 1bh ; "заглушка" для Tmr1_ovf
        nop
        reti
org 23h ; "заглушка" для UART
        nop
        reti
org 2bh
START:
; Инициализация МК ******************************************
        mov SP, #07h            ; стек сразу над банком 0
        setb PIN_CSEN           ; разрешаем дешифратор адреса, иначе клавиатура не выбирается
        mov DPTR, #ADR_KB
        clr A
        movx @DPTR, A           ; опускаем все столбцы: любое нажатие замкнёт строку на 0 и даст INT0
        setb IT0                ; INT0 ловим по спаду — одно нажатие = один вызов
        setb EX0                ; разрешаем прерывание от клавиатуры
        setb EA                 ; и прерывания вообще
        jmp $ ; %stop%

; KbScan — опрос матрицы клавиатуры 3x4 (вызывается из обработчика INT0).
; Столбцы опускаются в 0 по одному, по прочитанным строкам ищется единственная нажатая клавиша.
; Вход: нет. Выход: A — код клавиши по табл. 3 ТЗ (0…9, «Т» = 10, «С» = 11) или 0FFh,
; если не нажато ничего либо нажато две клавиши и больше. После опроса все столбцы снова в 0.
; Портит только A: PSW, DPTR, B, R2–R5 сохраняются в стеке.
KbScan:
        push PSW
        push B
        push DPL
        push DPH
        push AR2
        push AR3
        push AR4
        push AR5
        mov DPTR, #ADR_KB
        mov R4, #0FFh           ; R4 — найденный код; 0FFh — пока ничего не нашли
        mov R2, #0              ; R2 — номер текущего столбца (0 = Col1)
        mov R3, #11111110b      ; R3 — что пишем в столбцы: 0 только у опрашиваемого
KsCol:
        mov A, R3
        movx @DPTR, A           ; опускаем один столбец, остальные держим в 1
        movx A, @DPTR           ; читаем, какие строки он притянул к 0
        anl A, #(1 SHL KB_ROWS) - 1   ; оставляем только подключённые строки Row1…Row4
        xrl A, #(1 SHL KB_ROWS) - 1   ; переворачиваем: теперь 1 = клавиша на этой строке нажата
        jz KsNext               ; в этом столбце ничего — к следующему
        cjne R4, #0FFh, KsBad   ; клавиша уже была в другом столбце — нажато две
        mov B, A
        dec A
        anl A, B                ; x AND (x-1) = 0 только если нажата ровно одна строка
        jnz KsBad               ; две строки в одном столбце — ошибка
        mov A, B
        mov R5, #0              ; R5 — номер строки, ищем единственную единицу
KsRow:
        rrc A                   ; выталкиваем младшую строку в перенос
        jc KsHit                ; нашли нажатую строку
        inc R5
        sjmp KsRow
KsHit:
        mov A, R5
        mov B, #KB_COLS
        mul AB                  ; позиция клавиши в таблице = строка * 3 + столбец
        add A, R2
        mov DPTR, #KbCodes
```

```
        movc A, @A+DPTR         ; берём код клавиши по раскладке рис. 4 ТЗ
        mov R4, A               ; запоминаем, но опрос продолжаем — вдруг нажата ещё одна
        mov DPTR, #ADR_KB       ; возвращаем адрес клавиатуры
KsNext:
        mov A, R3
        rl A                    ; ноль переезжает на следующий столбец
        mov R3, A
        inc R2
        cjne R2, #KB_COLS, KsCol      ; пока не прошли все три столбца
        mov A, R4               ; итог: код единственной клавиши или 0FFh (ничего не нажато)
        sjmp KsEnd
KsBad:
        mov A, #0FFh            ; нажато больше одной клавиши
KsEnd:
        mov R4, A               ; придерживаем результат
        mov DPTR, #ADR_KB
        clr A
        movx @DPTR, A           ; все столбцы обратно в 0 — следующее нажатие снова даст INT0
        clr IE0                 ; пока перебирали столбцы, линия INT0 дёргалась — гасим ложный запрос
        mov A, R4
        pop AR5
        pop AR4
        pop AR3
        pop AR2
        pop DPH
        pop DPL
        pop B
        pop PSW
        ret

; Коды клавиш по строкам (Row1…Row4), в строке — Col1…Col3 (рис. 4 ТЗ, раскладка 3x4)
KbCodes:
        DB 1, 2, 3
        DB 4, 5, 6
        DB 7, 8, 9
        DB 0, 10, 11            ; «0», «Т», «С»
END
```

Программа 2 — файл «Судьбинин НА-код-2»

```
; Судьбинин Н.А., Аэ-21-23, 11, v1
$INCLUDE (vars.inc)             ; адреса, биты, константы варианта; в файл для робота mpscode вклеит его текст
DSEG AT 30h
; отдельных переменных нет: «голова», «хвост» и флаги — по адресам из vars.inc
CSEG AT 0h
        sjmp START
org 03h ; "заглушка" для INT0
        nop
        reti
org 0bh ; "заглушка" для Tmr0_ovf
        nop
        reti
org 13h ; "заглушка" для INT1
        nop
        reti
org 1bh ; "заглушка" для Tmr1_ovf
        nop
        reti
org 23h ; "заглушка" для UART
        nop
        reti
org 2bh
START:
; Инициализация МК ********************************************
        mov SP, #07h            ; стек сразу над банком 0
        setb PIN_CSEN           ; разрешаем дешифратор адреса — без него буфер не выбирается
        mov HEAD_L, #LOW(BUF_START)   ; «голова» — на первую ячейку буфера
        mov HEAD_H, #HIGH(BUF_START)
        mov TAIL_L, #LOW(BUF_START)   ; «хвост» — туда же: буфер пуст
        mov TAIL_H, #HIGH(BUF_START)
        setb F_EMPTY            ; отсчётов нет
        clr F_OVF               ; и ничего не терялось
        mov A, #55h             ; пример отсчёта
        call BufWrite ; %proc%
        jmp $ ; %stop%

; BufWrite — запись отсчёта в кольцевой буфер IDT7005.
; Буфер полон, когда «голова» догнала «хвост» и он не пуст (политика V: занято все BUF_SIZE ячеек);
; тогда самый старый отсчёт выбрасывается — «хвост» сдвигается вперёд, ставится F_OVF.
; Вход: A — отсчёт. Выход: отсчёт в ячейке бывшей «головы», «голова» на следующей ячейке,
; F_EMPTY = 0; F_OVF = 1, если затёрт самый старый отсчёт (иначе F_OVF не меняется).
; Портит A, B, DPTR, PSW.
BufWrite:
        mov B, A                ; отсчёт придержим в B, A нужен для сравнений
        jb F_EMPTY, BwPut       ; пустой буфер точно не полон
```

```
        mov A, HEAD_L
        cjne A, TAIL_L, BwPut   ; «голова» не на «хвосте» — место есть
        mov A, HEAD_H
        cjne A, TAIL_H, BwPut
        mov DPL, TAIL_L         ; буфер полон: выбрасываем самый старый отсчёт
        mov DPH, TAIL_H
        call BufNext            ; «хвост» на следующую ячейку
        mov TAIL_L, DPL
        mov TAIL_H, DPH
        setb F_OVF              ; отмечаем, что отсчёт потерян
BwPut:
        mov DPL, HEAD_L         ; адрес ячейки, куда пишем
        mov DPH, HEAD_H
        mov A, B
        movx @DPTR, A           ; кладём отсчёт в буфер
        call BufNext            ; «голова» на следующую ячейку
        mov HEAD_L, DPL
        mov HEAD_H, DPH
        clr F_EMPTY             ; теперь в буфере точно есть отсчёт
        ret

; BufNext — следующая ячейка кольцевого буфера.
; Вход: DPTR — адрес ячейки буфера. Выход: DPTR — адрес следующей; за последней ячейкой (BUF_END − 1)
; идёт первая (BUF_START). Портит A, PSW.
BufNext:
        inc DPTR
        mov A, DPL
        cjne A, #LOW(BUF_END), BnRet  ; младший байт не на границе — заворота нет
        mov A, DPH
        cjne A, #HIGH(BUF_END), BnRet ; старший тоже должен совпасть
        mov DPTR, #BUF_START    ; вышли за буфер — заворачиваем на начало
BnRet:
        ret
END
```

Программа 3 — файл «Судьбинин НА-код-3»

```
; Судьбинин Н.А., Аэ-21-23, 11, v1
$INCLUDE (vars.inc)             ; адреса, биты, константы варианта; в файл для робота mpscode вклеит его текст
DSEG AT 30h
X1:     DS 1                    ; цифра с клавиатуры (0 — Y2 пассивен)
X2:     DS 1                    ; отсчёт из буфера
CSEG AT 0h
        sjmp START
org 03h ; "заглушка" для INT0
        nop
        reti
org 0bh ; "заглушка" для Tmr0_ovf
        nop
        reti
org 13h ; "заглушка" для INT1
        nop
        reti
org 1bh                         ; Timer1 досчитал T2 — строб Y2 закончен
        setb PIN_Y2             ; первым делом снимаем строб, чтобы он не затянулся
        clr TR1                 ; таймер больше не нужен до следующего вывода Y2
        reti
org 23h ; "заглушка" для UART
        nop
        reti
org 2bh
START:
; Инициализация МК ********************************************
        mov SP, #07h            ; стек сразу над банком 0
        setb PIN_CSEN           ; разрешаем дешифратор адреса — без него регистр Y2 не выбирается
        setb PIN_Y2             ; строб Y2 в пассивной единице
        mov TMOD, #10h          ; Timer1 — 16-битный счётчик (режим 1), Timer0 не используется
        setb ET1                ; переполнение Timer1 заканчивает строб
        setb EA
        mov X1, #3              ; пример входных данных
        mov X2, #200
        call Y2Out ; %proc%
        jmp $ ; %stop%

; Y2Out — расчёт Y2 = (G + M + X1 + X2) mod 256, при X1 = 0 выдаётся Y2 = 0;
; значение пишется в регистр Y2, затем запускается строб PIN_Y2 = 0 длительностью T2.
; Процедура не ждёт конца строба: его снимает обработчик Timer1.
; Вход: X1, X2 (внутреннее ОЗУ). Выход: регистр Y2, PIN_Y2 = 0, Timer1 запущен на T2.
; Портит A, DPTR, PSW.
Y2Out:
        mov A, X1
        jz Y2Put                ; X1 = 0 — выход пассивный, выводим ноль
        add A, X2
        add A, #Y2_CONST        ; + (G + M); перенос за 8 бит отбрасывается — это и есть mod 256
Y2Put:
```

```
        mov DPTR, #ADR_Y2
        movx @DPTR, A           ; сначала значение в регистр — к началу строба оно уже на выходе
        clr TR1                 ; на случай, если прошлый строб ещё идёт — перезапускаем отсчёт
        mov TH1, #T2_RELOAD_H   ; загрузка на всю длительность T2
        mov TL1, #T2_RELOAD_L
        clr TF1                 ; убираем старый флаг переполнения, чтобы не было ложного конца
        clr PIN_Y2              ; начало строба
        setb TR1                ; и сразу пускаем отсчёт T2
        ret
END
```